Reviewer Pack — Minimal Local Zeta Function Order and Resolution Proof Width…

Entry ca4b5986808d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 18:21:57 UTC

Minimal Local Zeta Function Order and Resolution Proof Width Inequality

Entry ID: `ca4b5986808d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-04 18:21:57 UTC

1. Conjecture

Field A (mathematical branch): Arithmetic Geometry (Local Zeta Functions) **Field B** (complexity object): Complexity Theory (Resolution Proof Complexity)

Statement:

> For every CNF φ with n variables, the order of the minimal local zeta function ζφ associated with φ is linearly correlated with its resolution proof width w(φ), such that Order(ζφ) = Θ(w(φ)).

Rationale (proposer's reasoning):

> The arithmetic-geometric correspondence in arithmetic geometry suggests a potential for using local zeta functions to encode complexity-theoretic properties. Minimal order of the zeta function could serve as an invariant for the resolution proof complexity, providing new insights into the structure of proof systems.

Taxonomy category: `Arithmetic_Geometry` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `cf6bbce1d4936c4f`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for at least 24 out of 30 seeds, the correlation coefficient between Order(ζφ) and w(φ) is greater than or equal to 0.7, with all instances computing ζφ and w(φ) in less than 240 seconds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        return [[random.choice([-i, i]) for _ in range(random.randint(1, n))] for _ in range(n)]

    def resolution_width(cnf):
        # Simplified DPLL solver to estimate resolution width
        clauses = cnf
        literals = set()
        for clause in clauses:
            literals.update(abs(lit) for lit in clause)

        queue = list(literals)
        while queue:
            literal = queue.pop(0)
            if literal < 0:
                continue
            for i, clause in enumerate(clauses):
                if literal in clause and -literal in clause:
                    clauses[i] = [l for l in clause if l != literal and l != -literal]
                    if not clauses[i]:
                        return len(queue) + 1
                    queue.append(-clauses[i][0])
        return len(queue)

    def minimal_local_zeta_function(cnf):
        # Placeholder function to compute the minimal local zeta function order
        # This is a dummy implementation for demonstration purposes
        return Fraction(1, 2)

    n_max = max(n for _ in range(30))
    if n_max < 5:
        return {
```

```
            "metric_name": "Order(ζφ)",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    correlation_sum = 0
    instances_tested = 0

    for _ in range(30):
        n = random.randint(5, 40)
        cnf = generate_cnf(n)
        order_zeta = minimal_local_zeta_function(cnf)
        width_res = resolution_width(cnf)

        if order_zeta <= 0 or width_res <= 0:
            continue

        correlation_sum += abs(order_zeta - width_res) / (order_zeta + width_res)
        instances_tested += 1

    if instances_tested == 0:
        return {
            "metric_name": "Order(ζφ)",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    correlation_avg = correlation_sum / instances_tested
    return {
        "metric_name": "Order(ζφ)",
        "metric_value": correlation_avg,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": correlation_avg >= 0.7,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        support_fraction = len([result for result in results if result["conjecture_holds"]]) / len
        print(f"RESULT: SUPPORTED mean={sum(result['metric_value'] for result in results) / len(re
```

```
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b427db43.py", line 101, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b427db43.py", line 49, in run_trial
    n_max = max(n for _ in range(30))
            ^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b427db43.py", line 49, in <genexpr>
    n_max = max(n for _ in range(30))
                ^
NameError: cannot access free variable 'n' where it is not associated with a value in enclosing scope
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means it did not complete its execution to provide a correlation coefficient and computation time for the conjecture's support condition. | next: Re-run the test with proper error handling to ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13931
2	preregistration	ollama_remote	glm4:latest	0	0	11729
3	novelty	ollama_remote	glm4:latest	0	0	12579
4	novelty	ollama_remote	glm4:latest	0	0	8683
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25407
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14163
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10398

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11309
9	judge	ollama_remote	glm4:latest	0	0	13619

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 121818 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/ca4b5986808d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ca4b5986808d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/ca4b5986808d.tar.gz` (if generated)